

US Delivery Internship

Technical Interview — Task Round

Production-grade AI for Technical Support & TAM Teams

Duration 4 hours
(take-home)

Total marks 100

Format Individual submission

Deliverable GitHub repo +
Loom walkthrough

Brief & context

You are joining an AI team that builds internal tooling for two customer-facing units: **Technical Support** (tier-1 and tier-2 engineers resolving tickets) and **Technical Account Management** (TAMs who maintain strategic customer relationships). The team ships LLM-powered products used daily by hundreds of internal employees.

You have access to a mock dataset provided in the starter repo: 500 synthetic support tickets, 50 synthetic customer account summaries, and a product knowledge-base (Markdown docs). Use these exclusively — do not introduce external data.

Task 1 • Intelligent ticket triage agent

30 marks | Classification · Routing · Prioritisation

Build a triage pipeline that ingests an incoming support ticket (free-text) and produces a structured triage output without any human labelling.

Requirements

- Accept a raw ticket as input (text or JSON with subject + body).
- Classify the ticket into a product area, issue category, and urgency tier (P1-P4) with reasoning.
- Identify if the ticket matches any known issue pattern in the knowledge base and surface the relevant doc.
- Suggest a recommended responder team and a draft first-response message for the support agent.
- Expose this as a callable Python function or a lightweight REST endpoint (FastAPI acceptable).

LLM prompting

RAG / retrieval

Structured output

Python

Task 2 • TAM account health summariser

25 marks | Summarisation · Risk detection · Synthesis

TAMs spend 30+ minutes before each QBR manually piecing together account context. Build a tool that auto-generates a concise, actionable account brief from structured account data and ticket history.

Requirements

- Accept an account ID; pull the relevant account summary and last 90 days of tickets from the mock dataset.

- Produce a 3-section brief: executive summary (3–5 sentences), open risks & flagged issues, and recommended talking points for the TAM.
- Flag any tickets that suggest churn risk or escalation signals — justify each flag with a direct quote from the ticket.
- Output must be deterministic for the same input (control temperature, seed, or post-process to ensure this).

Multi-doc summarisation

Prompt chaining

Determinism

Output formatting

Task 3 • Evaluation harness

20 marks | LLM evals • Quality gates • Regression testing

Any production AI solution needs an eval framework. Write an evaluation harness that tests both Task 1 and Task 2 outputs systematically.

Requirements

- Define at least 5 test cases per task with expected outputs or acceptance criteria (LLM-as-judge, rule-based, or both).
- Implement a scoring function that reports pass/fail and a quality score (0–1) per test case.
- Produce a summary report (JSON or Markdown table) showing results across all test cases.
- Include at least one adversarial test case per task (e.g. ambiguous ticket, incomplete account data).

Eval design

LLM-as-judge

Test coverage

Reporting

Task 4 • Design note

15 marks | Written, ~600 words | Included in your README or a separate doc

A short written section included in your repo's README or as a separate Markdown file. No slides required. Address all four points below.

Failure modes:	What are the top 3 ways your solution could fail in production? How would you detect and mitigate each?
Latency vs quality:	Describe one concrete trade-off you made between response speed and output quality. What would you change if latency were the hard constraint?
Data sensitivity:	Ticket and account data may contain PII. How does your design handle or avoid leaking sensitive data to external APIs?
Scaling:	How would this solution behave with 10× the ticket volume? What breaks first?

Bonus • Up to 10 marks

Not all bonus marks can be earned — pick one or two areas of genuine strength.

+5 Thin UI demo (Streamlit / Gradio) that a non-technical TAM could actually use.

- | | |
|-----------|--|
| +3 | Streaming output implemented and demonstrated in Task 1 or 2. |
| +2 | Automated CI step that runs the eval harness on every commit (GitHub Actions). |
| +2 | Prompt versioning: each prompt is tracked with a version identifier and changelog. |

Submission checklist

- Public or shared **GitHub repo** containing all code and a top-level README.
- README includes: setup instructions, a sample run for each task, and the design note (or link).
- **Loom video** (3–6 min) walking through the code, a live demo of Tasks 1 & 2, and eval results.
- Eval harness report included as a file in the repo (*eval_report.json* or *eval_report.md*).
- *.env.example* file showing required environment variables — never the real values.

Submission deadline is 48 hours from when the starter repo link is shared. Late submissions are not reviewed.

Automatic disqualifiers

API key or credential committed to the repository in any form.

Code that does not run from a clean *pip install -r requirements.txt* plus a single entry-point command.

Evidence of plagiarism or wholesale copying from another candidate's submission.

Using data outside the provided mock dataset (live scraping, real customer data, etc.).

Loom walkthrough absent or under 3 minutes.

Good luck.